

Adatbázis-kezelés II.

Triggerek

Rostagni Csaba

2026. január 18.

Ezen az órán... I

- 1 Prepared Statements
- 2 Hibajelzések
- 3 Triggerek - TRIGGERS

Tartalom I

1 Prepared Statements

Prepared Statements

- Dinamikus SQL / paraméterezett lekérdezések
- Sok esetben ugyanaz a lekérdezés, csak 1-2 paraméterben különbözik
- A szerver oldalon is elkészíthetjük a lekérdezést, a változó részek helyére placeholderek kerülhetnek
- SQL injection elleni védelem
- Összeállítható lekérdezés

Linkek:

- MySQL dokumentáció: Prepared Statements
- MySQL dokumentáció: A szabványtól eltérő megjegyzés

Prepared Statements

- **PREPARE** - előkészítés a futtatáshoz
- **EXECUTE** - végrehajtás
- **DEALLOCATE PREPARE** - felszabadítás

A termékek tábla

id	nev	kategoria	netto	penznem	afa
1	4K TV	tv	499	EUR	0.19
2	Mobil 32GB	mobil	299	EUR	0.19
3	Mobil 128GB	mobil	679	EUR	0.19
4	Olcsó laptop	laptop	269	EUR	0.19
5	Drága laptop	laptop	1729	EUR	0.19
6	Könyv	könyv	<i>NULL</i>	<i>NULL</i>	<i>NULL</i>

Prepared Statement example

Mezők meghatározása

MySQL

```
SET @mezo = 'netto';  
SET @sql = CONCAT('SELECT MAX(', @mezo, ') FROM termék');  
  
PREPARE stmt FROM @sql;  
EXECUTE stmt;
```

- Összefűzéssel olyan elemekre is hivatkozhatunk, amire amúgy nem lehetne (mező neve, tábla neve)

Termékek száma X kategóriában

Hány termék található a(z) X kategóriában? Általános SQL lekérdezés előkészítése:

MySQL

```
SET @sql = 'SELECT COUNT(*) FROM termekek WHERE  
↪ `kategoria` = ?';  
PREPARE stmt FROM @sql;
```

Hány termék található a **könyv** kategóriában?

MySQL

```
SET @kategoria = 'könyv';  
EXECUTE stmt USING @kategoria; // változónak kell lennie!
```

Hány termék található a **mobil** kategóriában?

MySQL

```
SET @kategoria = 'mobil';  
EXECUTE stmt USING @kategoria;
```

Több változó használata

Hány termék van az X kategóriában? Ami Y-nál drágább?
Általános SQL lekérdezés előkészítése:

MySQL

```
SET @sql = 'SELECT COUNT(*) FROM termek WHERE  
↪ `kategoria` = ? AND `ar` > ?';  
PREPARE stmt FROM @sql;
```

Hány termék van a **laptop** kategóriában? Ami **500**-nál drágább?

MySQL

```
SET @kategoria = 'laptop';  
SET @minar = 500;  
EXECUTE stmt USING @kategoria, @minar;
```

- Több termék esetén a kérdőjelek sorrendjében kell megadni a változókat.

Tartalom I

2 Hibajelzések

Hibajelzés - SIGNAL

- **Adatérvényesítés** data validation: Célszerű ellenőrizni a bemenetet.
- Hibajelzéseket hibakódok segítségével adhatunk.

Hibakód használatának általános leírása:

```
SIGNAL SQLSTATE [VALUE] állapot_ertek  
[  
    SET MESSAGE_TEXT = uzenet  
    [  
        , MYSQL_ERRNO = mysql_hibakod  
    ]  
]
```

Hibajelzés - SIGNAL

MySQL

```
SIGNAL SQLSTATE VALUE '22003'  
SET MESSAGE_TEXT = 'Nem megfelelő érték',  
MYSQL_ERRNO = 1264;
```

- A felhasználók által generált hibák számát a MariaDB és a MySQL a '45000' SQLSTATE osztályt ajánlja.
- A 'HY000' az általános hibakód.

Linkek:

- <https://mariadb.com/kb/en/sqlstate/>
- <https://mariadb.com/kb/en/mariadb-error-codes/>
- <https://dev.mysql.com/doc/mysql-errors/8.0/en/server-error-reference.html>
- <https://en.wikipedia.org/wiki/SQLSTATE>

Tartalom I

3 Triggerek - TRIGGERS

Triggerek - TRIGGERS

A triggerek olyan névvel ellátott kódrészletek, melyek automatikusan végrehajtnak, egy megadott lekérdezés (INSERT, UPDATE vagy DELETE) hatására.

- MySQL 5.0-tól használhatjuk
- MySQL-ben csak a sor szintű triggerek támogatottak (minden egyes módosított sorra végrehajtnak)
- A régi értékeket az OLD kulcsszóval, az új értékeket a NEW kulcsszóval érhetjük el.

Triggerek általános megadása

```
CREATE TRIGGER trigger_neve  
{ BEFORE | AFTER } {INSERT | UPDATE | DELETE }  
ON tabla_neve  
FOR EACH ROW  
    ...  
BEGIN
```

Trigger példa I.

Az "emp" tábla "ename" mezőjébe csak nagybetűs értékeket lehessen felvinni!

MySQL

```
DELIMITER //  
CREATE TRIGGER emp_before_insert  
BEFORE INSERT ON emp  
FOR EACH ROW  
BEGIN  
    SET NEW.ename = UPPER(NEW.ename);  
END //
```

Figyelem!

Mivel az adatot most szűrjük be az adatbázisban itt az OLD használatának nincs értelme.

Trigger példa I. próba

MySQL

```
INSERT INTO `emp` (`empno`, `ename`)  
VALUES ('100', 'laci');
```

A trigger beszúráskor aktiválódik, így LACI fog az adatbázisba bekerülni.

Trigger példa II.

A fizetésemelés maximum 10% lehet! Különben maradjon a régi!

```
DELIMITER //  
CREATE TRIGGER emp_before_update_1  
BEFORE UPDATE ON emp  
FOR EACH ROW  
BEGIN  
    IF NEW.sal > OLD.sal * 1.1 THEN  
        SET NEW.sal = OLD.sal;  
    END IF;  
END //
```

logikai hiba

Trigger példa II. próba

MySQL

```
UPDATE `emp` SET `sal`= 10000 WHERE empno = 7369;
```

Figyelem!

Látszólag jó ötletnek tűnik, DE aki beírja az adatot nem biztos hogy észreveszi, hogy az érték nem változott, ugyanis mindeféle figyelmeztetés nélkül lefut.

Trigger példa III.

A fizetésemelés maximum 10% lehet! Különben hiba!

MySQL

```
DELIMITER //  
CREATE TRIGGER emp_before_update_2  
BEFORE UPDATE ON emp  
FOR EACH ROW  
BEGIN  
    IF NEW.sal > OLD.sal * 1.1 THEN  
        SIGNAL SQLSTATE 'HY000'  
        SET MESSAGE_TEXT = 'Fizu emelés MAX 10%!';  
    END IF;  
END //
```

Trigger példa III. próba

MySQL

```
UPDATE `emp` SET `sal`= 10000 WHERE empno = 7369;
```

- A trigger módosításkor aktiválódik
- Mivel a 7369-es azonosítójú személy jelenleg 800 USD fizetéssel rendelkezik az 1000 USD érték meghaladja a 10%-os emelés határt,
- Így hibaüzenettel értesít az adatbázis a módosítás helyett

Trigger példa IV.

Az emp táblából törlés után logoljon!

MySQL

```
DELIMITER //  
CREATE TRIGGER emp_after_delete  
AFTER DELETE ON emp  
FOR EACH ROW  
BEGIN  
    INSERT INTO log  
    VALUES( CONCAT(OLD.empno, ' azonosítójú dolgozó törölve')  
END //
```

Trigger példa IV. próba

MySQL

```
DELETE FROM `emp` WHERE empno = 7369;
```

- A trigger törléskor aktiválódik
- A log táblába beszúráásra kerül: "7369 azonosítójú dolgozó törölve"

Triggerek listázása

A összes trigger listázása az aktuálisan kiválasztott adatbázisban:

```
SHOW TRIGGERS;
```

MySQL

A összes trigger listázása a megnevezett adatbázisban:

```
SHOW TRIGGERS IN empdept;
```

MySQL

Keresés a név kezdete szerint:

```
SHOW TRIGGERS IN empdept LIKE 'emp%'
```

MySQL

Triggerek törlése

Trigger törlése:

```
DROP TRIGGER emp_before_insert;
```

MySQL

Trigger törlése ha létezik:

```
DROP TRIGGER IF EXISTS emp_before_insert;
```

MySQL